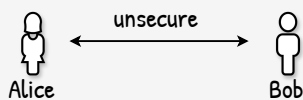


Cryptography II: Asymmetric Crypto, DH, Signatures, Certificates and Web of Trust

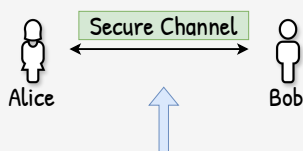
Taller de Seguridad
Ing. Martin Di Paola - Fiuba

El escenario es simple -> Alice y Bob quieren comunicarse a través de un canal inseguro donde Eve (el adversario) puede espiar y adulterar los mensajes.

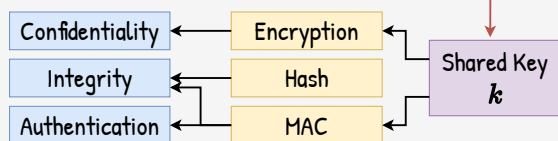
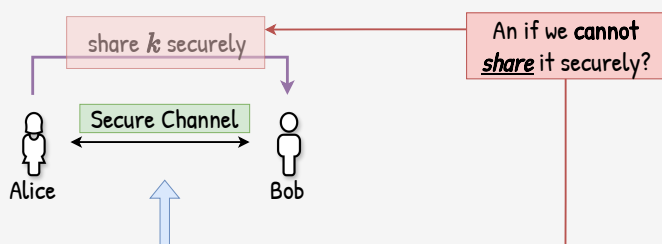
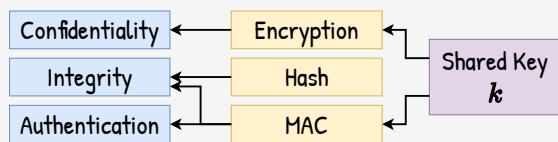
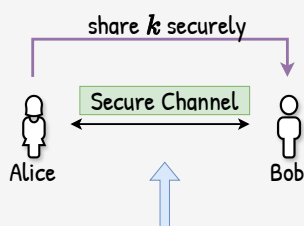
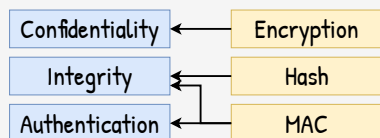
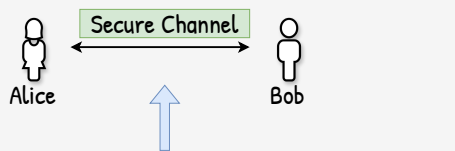


La solución es un *secure channel* que tiene 3 propiedades:

- confidencialidad: Eve no puede ver los mensajes enviados
- integridad: Eve no puede modificarlos sin que Alice/Bob lo detecten
- autenticidad: Eve no crear mensajes falsos (que parezcan venir de Alice/Bob) sin que Alice/Bob lo detecten



Confidentiality
Integrity
Authentication



Y para la construcción de un *secure channel* tenemos 3 primitivas:

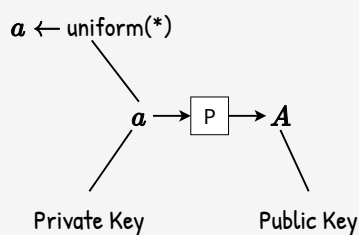
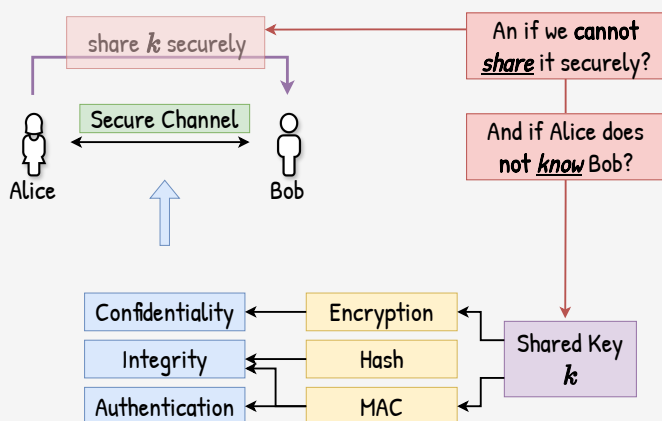
- encriptacion: si esta bien usado nos dara confidencialidad
- hash: nos dara integridad
- HMAC: que, junto con un hash, nos dara integridad y autenticidad

Hay muchas implementaciones distintas de estas primitivas criptográficas pero la mayoría (y las más rápidas y eficientes) usan una única clave secreta.

Dicha clave debe ser compartida por Alice y Bob.

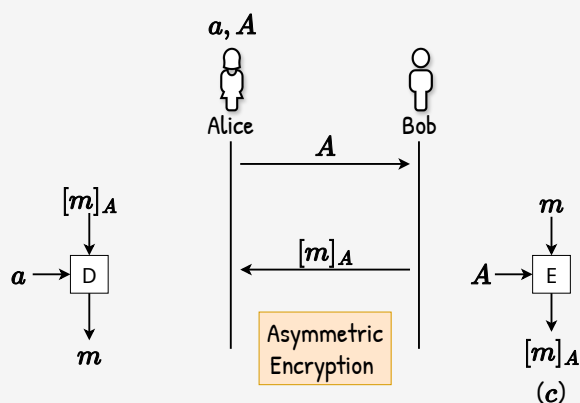
Y si Alice y Bob se conocen en persona, pueden intercambiar físicamente la clave en algún lugar (un café o una plaza)

Pero y si no? Y si no pudieran intercambiar la clave compartida de forma segura?



Public Key
Derivation

Given A is not
possible to find a



Y si no conocieras a la otra persona?

O a caso vos conoces y te juntaste con Mr Google e intercambiaste una clave con él?

Asumir que Alice y Bob pueden intercambiar y compartir una clave secreta no escala.

Alice genera una clave secreta (privada) a de forma completamente al azar. De ella, luego deriva otra clave, pública, A .

La derivación es tal que, si un adversario obtiene A , el no puede aprender nada de la clave a (de ahí q la primera es pública y la segunda privada).

Esta es la base de toda una rama de criptografía: la criptografía asimétrica.

Hay muchos esquemas de clave pública/privadas y derivaciones, algunos con más o menos limitaciones, con más o menos utilidad. No vamos a entrar en detalle en esto pero tenelo muy presente. La criptografía asimétrica requiere un especial cuidado y saber bien que estas haciendo y para que.

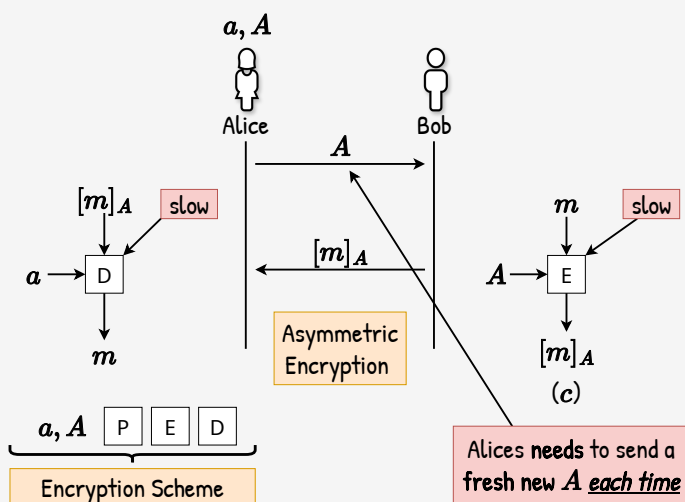
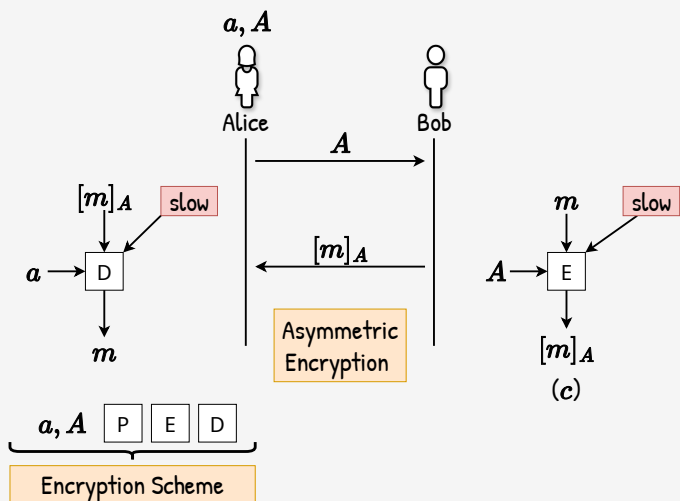
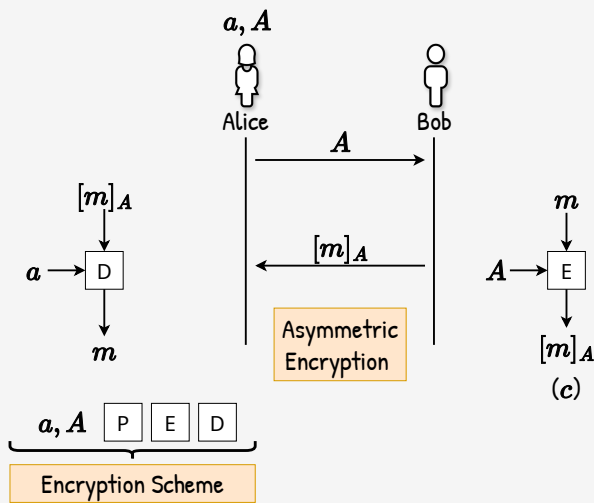
Alice le envía a Bob su clave pública A . Eve no puede hacer nada con esto ya q no puede reconstruir la privada a partir de la pública.

Bob usa A para encriptar un mensaje y se lo pasa a Alice.

Y acá está la magia: Bob usa la pública para encriptar pero la pública no puede ser usada para descryptar. Así q Eve, a pesar de saber A no le sirva de nada!

Es con la privada (a) con la q se puede descryptar, y como solo Alice la tiene, solo ella puede ver el mensaje.

De ahí el nombre: encriptación asimétrica



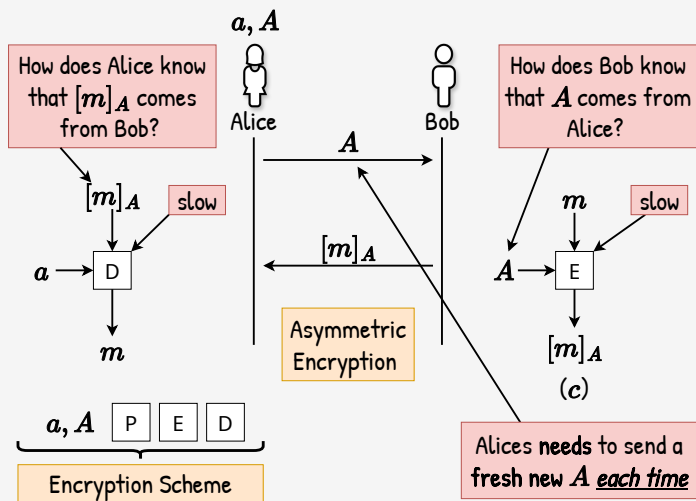
En la criptografía asimétrica se habla de “esquemas de encriptación” donde el esquema no es sólo la función de encriptación E y desencriptación D sino que incluye también a como se elige las claves privadas y como se derivan las públicas.

La encriptación es segura **solo** si se derivan y usan las claves segun el esquema. Usar claves de otros esquemas de encriptación no funciona.

Pero, esta magia tiene un precio. Los esquemas asimétricos son lentos (tanto en la derivacion de claves, como en la encriptación/desencriptación y en otras operaciones)

Y encima, como no podemos reutilizar claves para encriptar, Alice debería generar un nuevo par clave pública/privada por cada mensaje.

Escala mal.

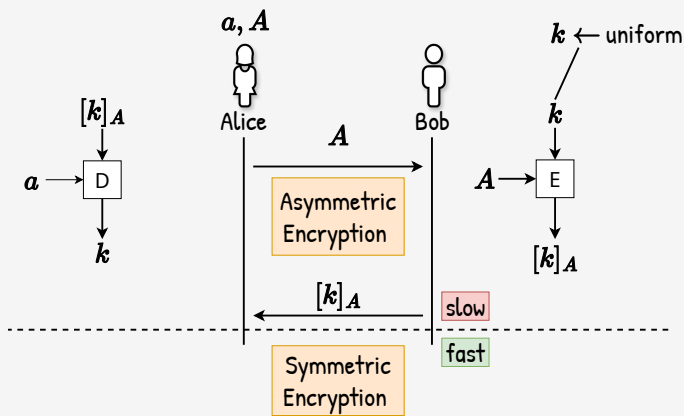


Y como sabe Alice q el mensaje encriptado vino de Bob? Eve podria haberlo creado (ya q para encriptar se necesita solo la pública A q ella ya tiene).

En el caso simétrico uno sabía que solo Alice y Bob compartian un secreto k , pero esto ya no es asi.

La primer solución es simple: Alice envia a Bob A pero Bob no la usa para encriptar un mensaje cualquiera. Él genera una clave k es se la pasa a Alice encriptada.

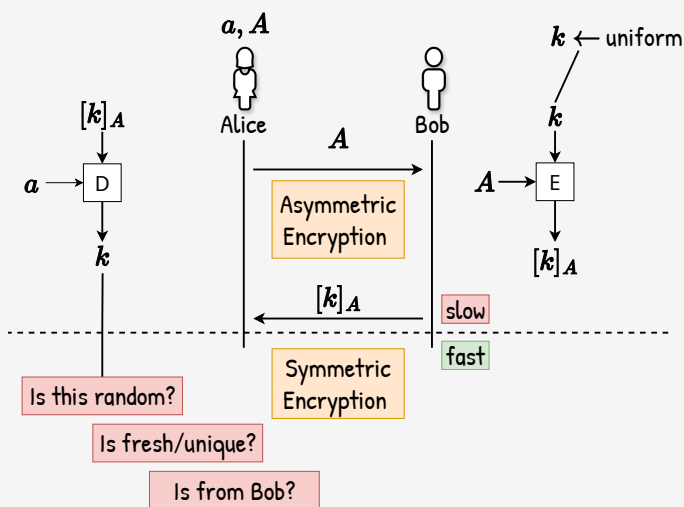
A partir de ahí, tanto Alice como Bob tiene acceso a una clave secreta compartida k y pueden implementar un canal seguro con cripto simétrica.

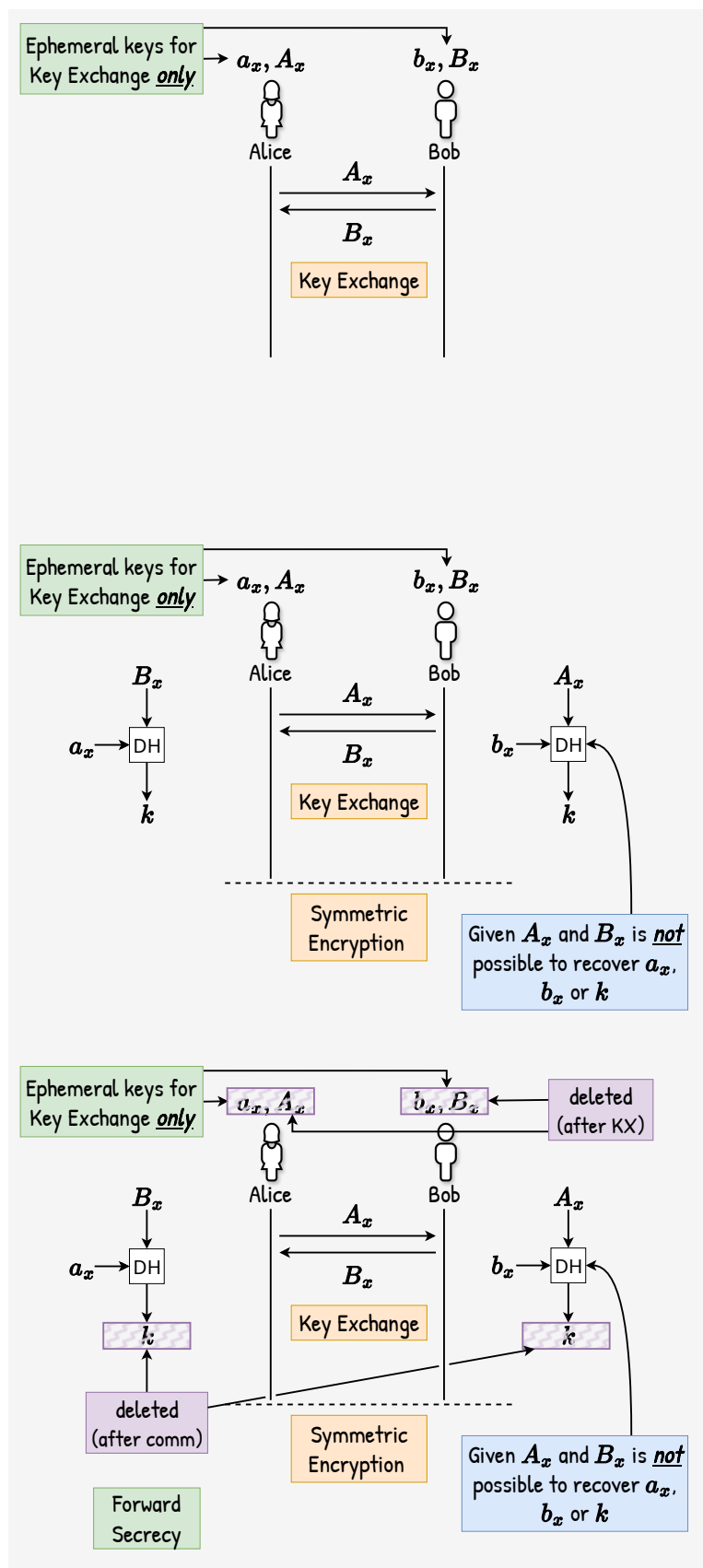


Es un hack no libre de desventajas:

- como puede saber Alice que k fue elegida al azar por Bob?
- como sabe que k vino de Bob?

Y si Alice y Bob quisieran tener otra comunicación otro dia, aun seguimos requiriendo que Alice genere un nuevo par pública/privada para no repetir claves.





Vamos por una solución más segura: un key exchange. Alice y Bob generan un par público/privado cada uno e intercambian sus públicas.

Cada uno toma su privada y la pública del otro y de ellas derivan un secreto k .

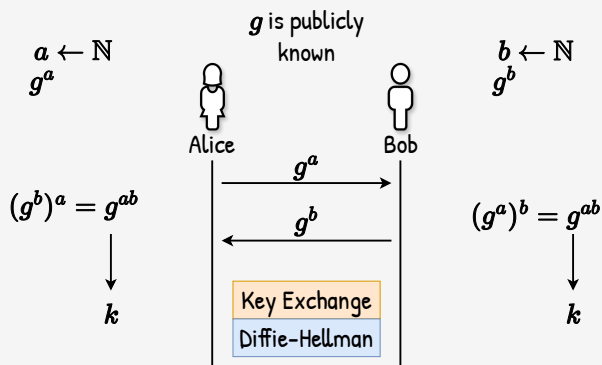
La magia está, aunque no lo creas, en que Alice y Bob van a generar el *mismo* secreto k !

Y sin embargo, Eve, aún con acceso a las 2 públicas, no puede aprender nada ni de las privadas ni del secreto compartido.

Los pares público/privados son generados y usados solo para el key exchange (KX). Si ahora Alice y Bob se deshacen de dichos pares, nadie puede reconstruir k .

Si cuando terminan de hablar ambos también borran k , nadie puede descryptar los mensajes, no importa si Eve va a la casa de Alice y la amenaza. Ya no hay nada q se pueda hacer para reconstruir ningún secreto ni descryptar nada.

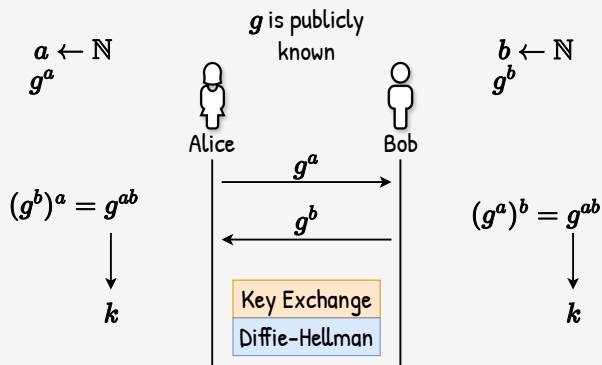
Esto se llama *forward secrecy* (secreto en adelante).



Y como es esta magia? La primitiva DH es **Diffie-Hellman** en honor a los 2 autores padres, no solo de este esquema de KX sino también, junto con Merkle, de la criptografía asimétrica.

Se toma un número entero g conocido por todos. Alice y Bob generan cada uno un número entero aleatorio secreto (a y b) y derivan un número público g^a y g^b .

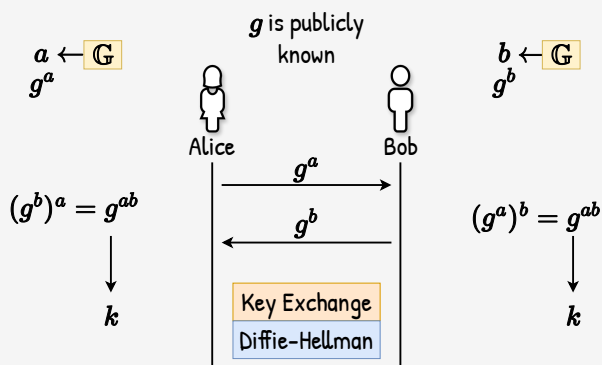
Los intercambian y ambos toman el publico del otro y lo elevan al número privado. Dado que $(g^a)^b = g^{ab} = (g^b)^a$, ambos llegan al mismo numero secreto k .



Te das cuenta que esto tiene una falla fatal: dado g^a es trivial encontrar el numero secreto a . Solo basta con calcular el logaritmo en base g !

Se supone que de lo público no se puede obtener nada de lo privado!

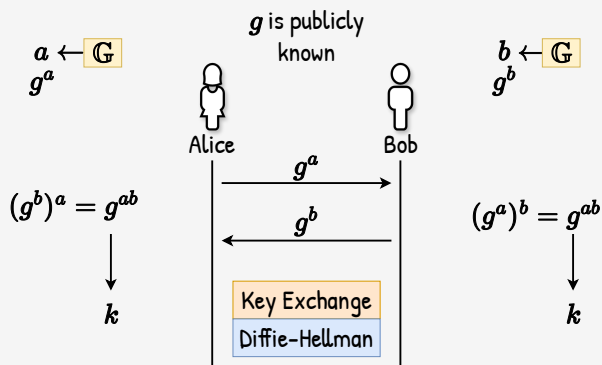
But given g^a , it is trivial to recover a with just $\log_g(g^a)$!!



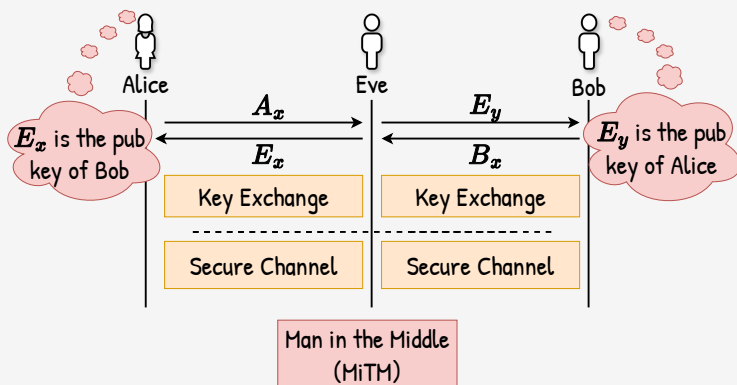
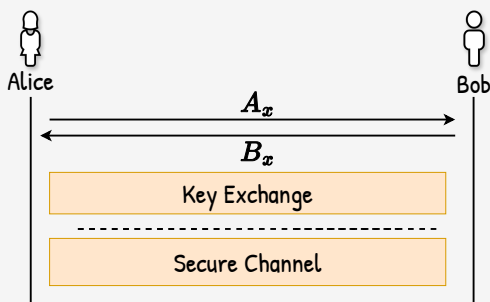
Resulta que este truquito matemático $(g^a)^b = g^{ab} = (g^b)^a$ aplica a *otros* grupos de numeros, no solo los números naturales.

But given g^a , it is trivial to recover a with just $\log_g(g^a)$!!

True for $\mathbb{N}$, but the same math trick works for another groups G



But given g^a , it is <i>trivial</i> <u>to recover</u> a with just $\log_g(g^a)$!!	True for $\mathbb{N}$, but the same math trick works for <u>another groups</u> $\mathbb{G}$	And for some groups $\mathbb{G}$ $\log_g(g^a)$ is <i>really hard</i> <u>to compute</u> (for now)
---	--	--



Y el truco esta en que para ciertos grupos $\mathbb{G}$, es muy difícil computar el logaritmo pero es muy fácil hacer la exponenciación.

O sea q para Alice y Bob es fácil hacer $(g^b)^a$ y $(g^a)^b$ pero para Eve es muy difícil encontrar a o b o k con solo g^a y g^b .

Que tan difícil es un problema abierto y al menos para ciertos grupos $\mathbb{G}$, por ahora, es lo suficientemente difícil.

En su paper de 1976, Diffie y Hellman usaron como $\mathbb{G}$ los naturales 1 a p (con p siendo un número primo muy grande). Hoy en dia se usan curvas elipticas como $\mathbb{G}$ (por ser más difícil de computar el logaritmo, al menos, por ahora).

Resumen:

- con KX (DH) Alice y Bob generan un secreto compartido k sin conocerse ni juntarse personalmente.
- con k luego establecen un canal seguro

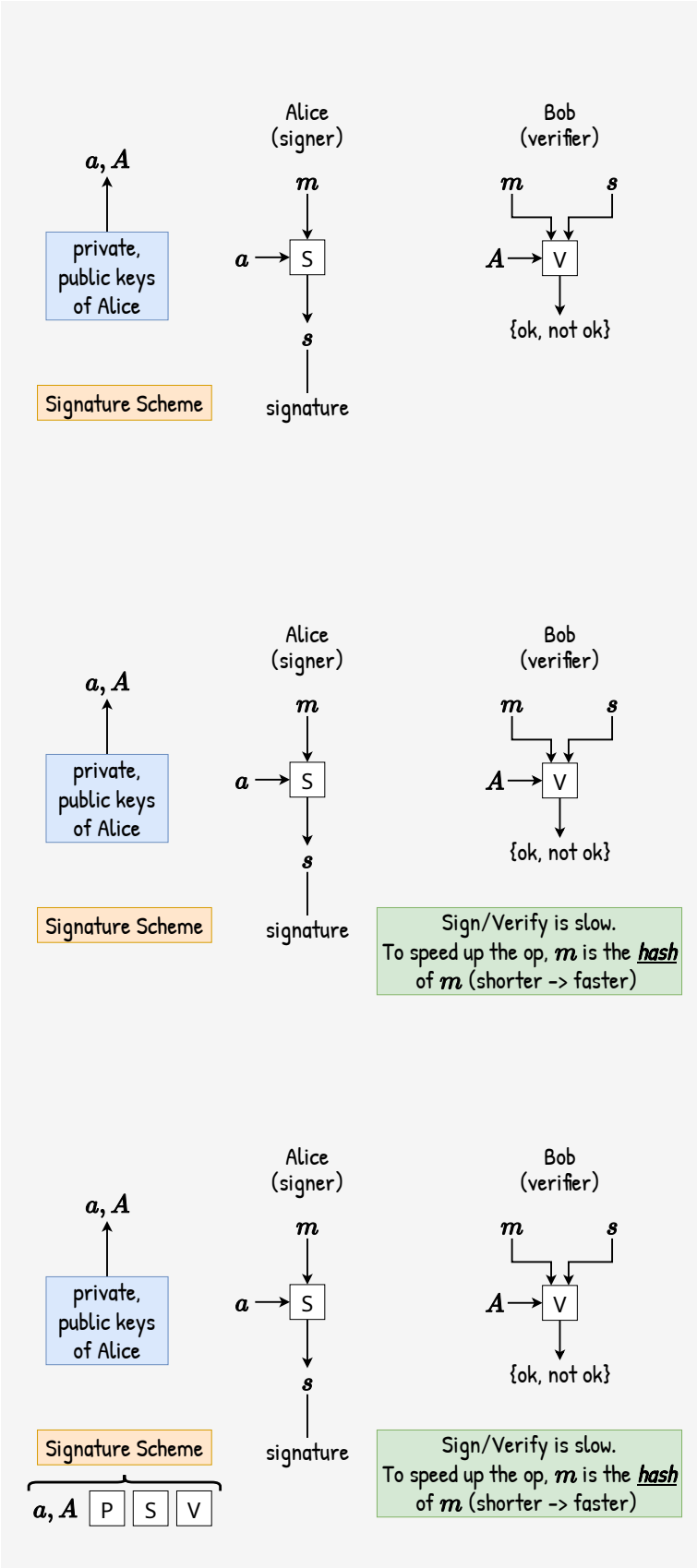
Pero...

Nada impide a Eve ponerse en el medio: hacia Alice, Eve se hace pasar por Bob; hacia Bob, ella se hace pasar por Alice.

Alice y Bob hacen KX y luego un canal seguro solo que lo hacen con Eve sin darse cuenta.

Entonces Eve tiene acceso completo a los mensajes.

El DH no nos da autenticidad ni forma de comprobar quien es quien.



Un par público/privado también se pueden usar para autenticar, lo que se llama un esquema de firmado.

Este tiene 2 operaciones:

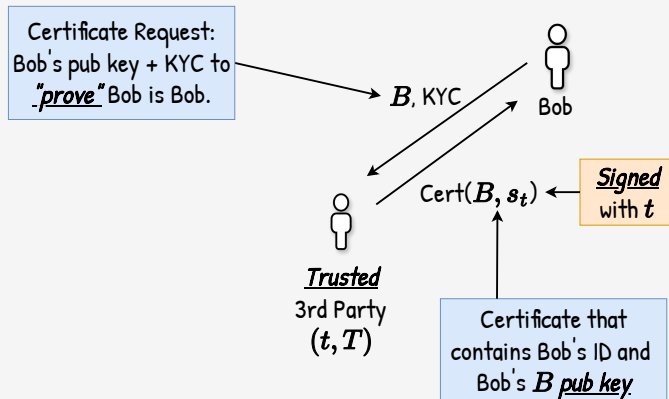
- firmar: Alice usa su clave privada para firmar un mensaje
- verificar: Bob usa la pública de Alice para verificar q el mensaje no fue ni adulterado ni falsificado por Eve

Como estas operaciones son lentas, en la practica lo q se firma/verifica no son los mensajes sino sus hashes (funciona similar a las MACs para los casos de clave compartida).

Al igual que pasa con la encriptación asimétrica, hay un “esquema de firmado” que dice como deben generarse los pares públicos y privados, como deben firmarse (S) y como verificarse (V).

Cualquier error y el sistema se vuelve inseguro. Por ejemplo, algunos esquemas de firmado requieren que no se reutilice la misma clave privada para firmar: firmar 2 veces revela la privada al atacante! En cambio, otros esquemas permiten firmar múltiples veces.

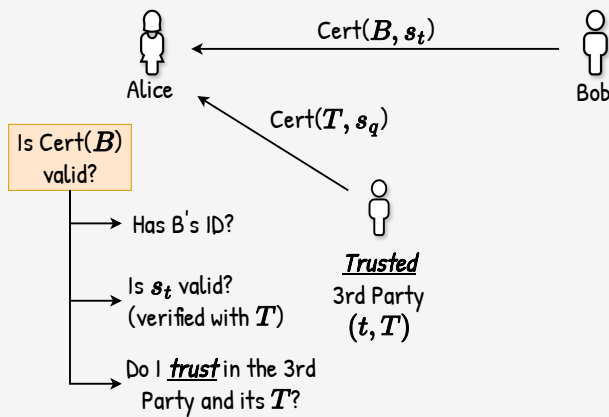
El esquema no dice que función de hash se puede usar así que aquí hay flexibilidad para la elección (mientras que la función de hash sea criptográficamente segura en sí misma).



Ahora, como puede saber Alice que B es la pública de Bob?

Bob va a pasarle a un 3rd party de confianza su pública B y el material q acredite que Bob es realmente Bob (KYC - Know Your Client)

El 3rd party de confianza, va a verificar los datos y si esta todo OK, le entregara a Bob un certificado (digital). Entre otras cosas el certificado tiene la identidad y la clave pública de Bob (B) y esta firmado por la 3rd party.

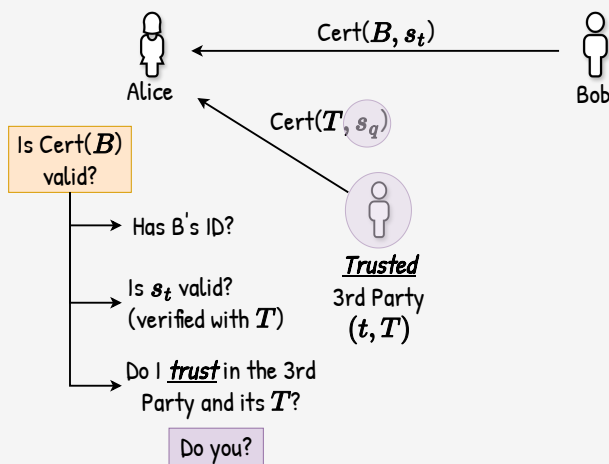


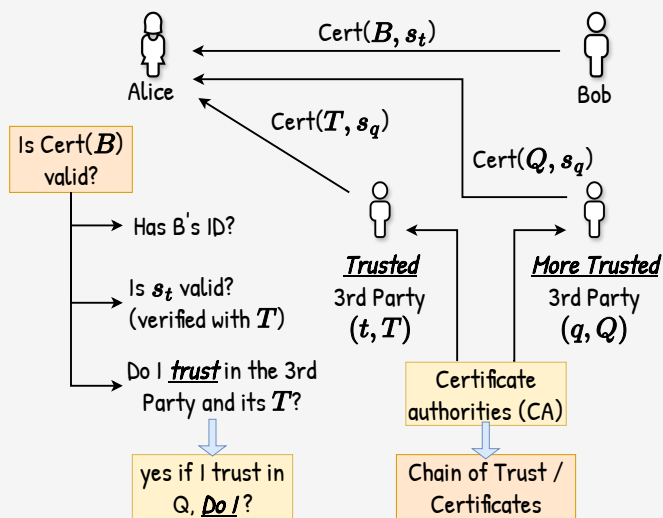
Cuando Alice quiere hablar con Bob, este le pasa su certificado. Alice lo verifica viendo:

- es el ID de Bob?
- el certificado es válido?
- la firma coincide (fue firmado por el 3rd party de confianza y no fue falsificada)?

Con el certificado de Bob, Alice puede estar segura de que esta hablando con él y no con Eve y que tiene en su mano la pública de Bob y no otra cosa.

Pero todo se basa en supuesto: Alice, confia en que el 3rd party **es** de confianza realmente? Como sabe que la pública T viene de él?



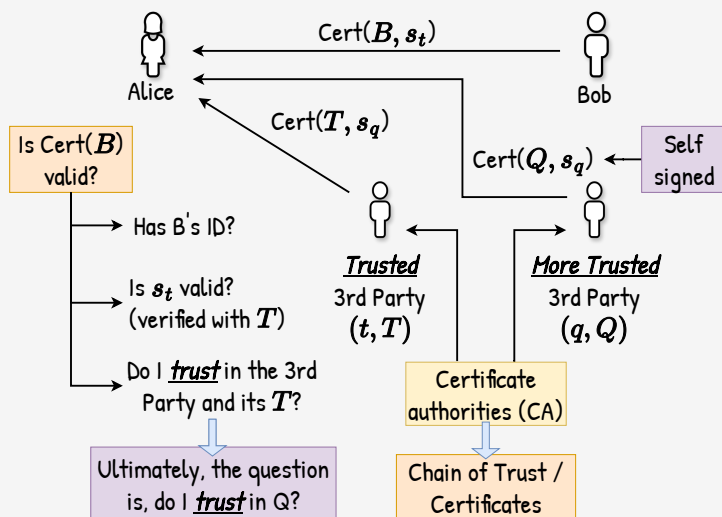


Alice puede pedirle al 3rd party q le entregue su certificado. Con él, Alice puede verificar la identidad y obtener la pública T y con ella, verificar el certificado de Bob.

Pero quien certificó y firmó el certificado de la 3rd party de confianza? Obvio! Otro 3rd party de *más* confianza!

Alice puede confiar en B por q confía en T. Pero se puede confiar en Q?

Estos 3rd parties son las llamadas autoridades certificadoras (CA) y forman una cadena de confianza.

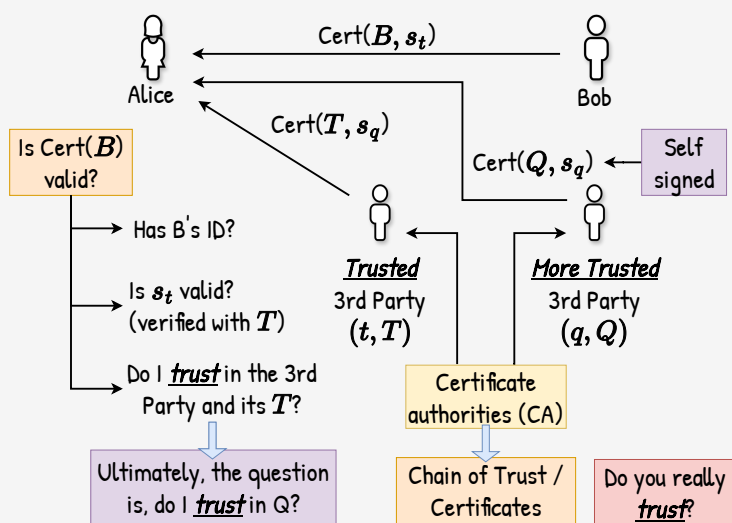


Y quien firma el certificado de Q? Al final de la cadena habrá una CA que se firme a sí misma (un self-signed certificated).

En última instancia, toda la seguridad reside si se puede confiar en él y en toda la cadena.

A nivel criptográfico, las firmas te garantizan integridad y autenticidad. Eve no puede hacer un MiTM.

Pero la criptografía no te garantiza q algun CA sea *shady* y venda al mejor postor certificados truchos (como un certificado "a nombre de Bob" pero con la pública de Eve)



Vos confias?

Y la respuesta es sí, por que toda la Internet usa este esquema. Cada vez que entras a un sitio <https://> tu browser usa certificados.

Pero ahora q sabes el detras de escena, realmente confias?



Alice

Owner's judgement			
Keys	Verified	judgement	Signed by
A	full	full	s_a



Alice

Owner's judgement			
Keys	Verified	judgement	Signed by
A	full	full	s_a
B	none	none	$s_b \quad s_d \quad s_e$



Bob



Alice

Owner's judgement			
Keys	Verified	judgement	Signed by
A	full	full	s_a
B	none	none	$s_b \quad s_d \quad s_e$



Bob

Alice does **not** know, for real, that **B** belongs to Bob and has no way to verify it.

Encaramos el problema de confianza desde otro ángulo.

Alice va a mantener un registro de todas las públicas que ella se encuentre y se va a preguntar si pudo verificar q dicha pública le pertenece a su dueño o no.

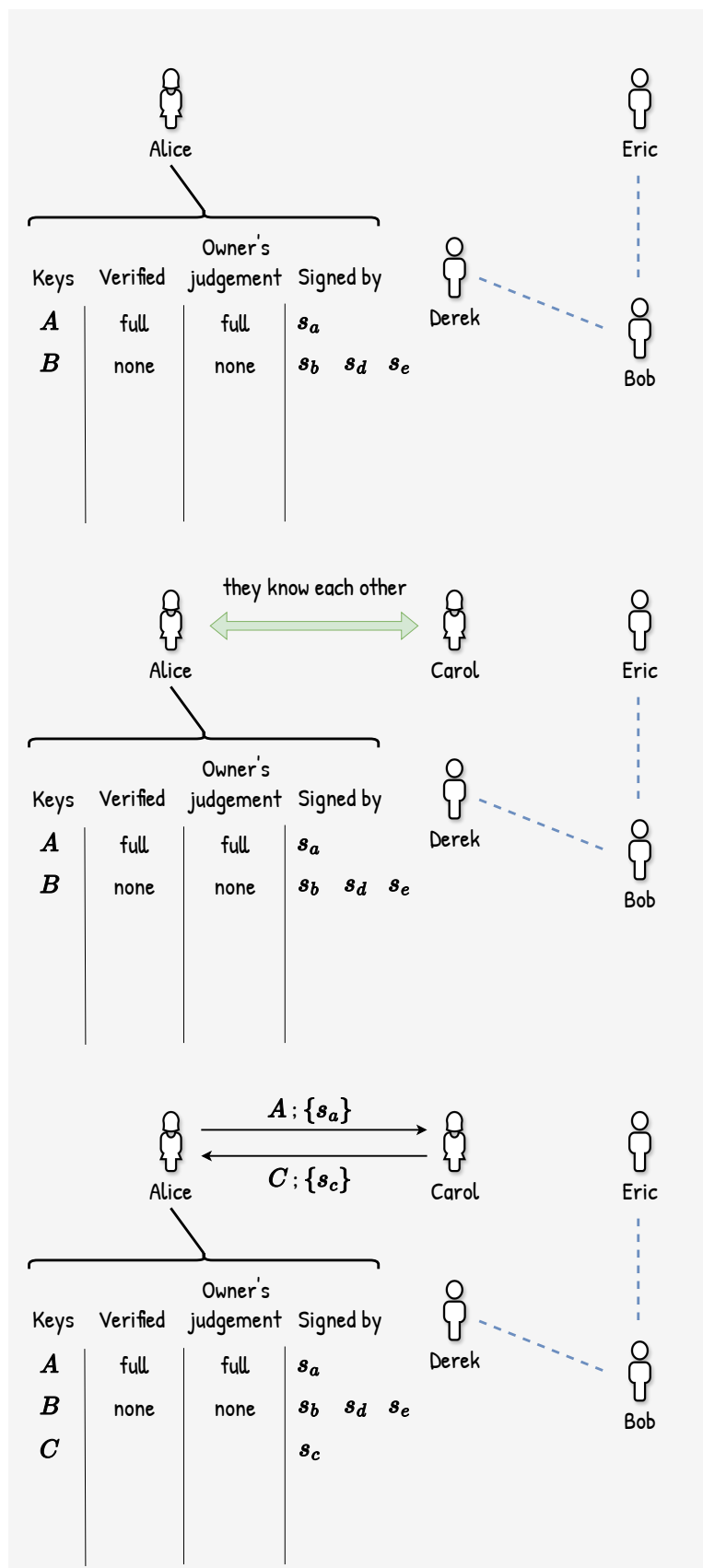
Se va a preguntar que tan confiable es la otra persona para hacer sus propias verificaciones (confia Alice en el juicio del otro?).

Inicialmente, Alice solo tiene su propia pública **A** y confia en ella completamente (obvio, Alice sabe que Alice la generó!)

Alice quiere hablar con Bob y consiguió su pública **B** pero ella no pudo verificar que **B** vino realmente de Bob.

Lo que sí sabe es que **B** fue firmada por 3 identidades: **b** (Bob), **d** y **e**. Asi que hay otras personas ahi afuera q conocen y verificaron la identidad de Bob.

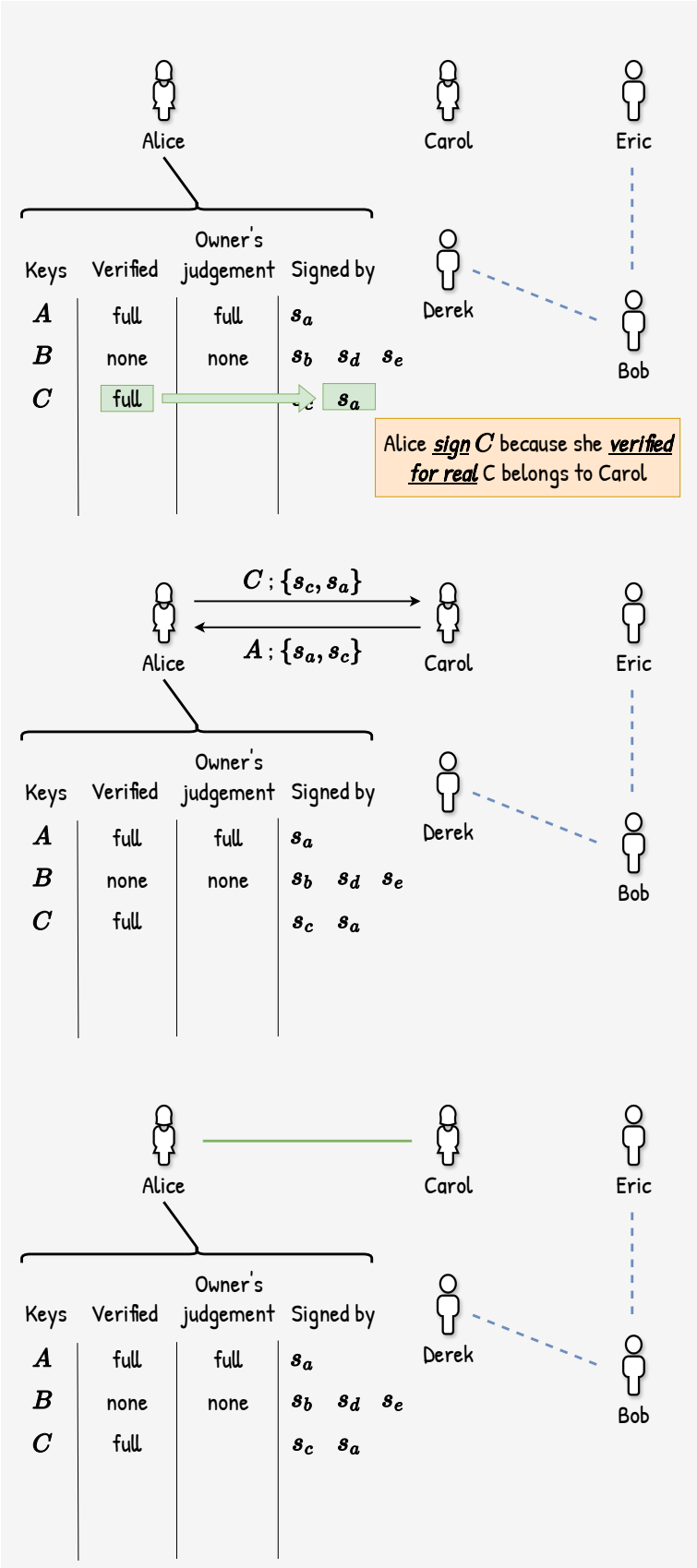
Pero no te dice mucho. Alice no sabe ni quienes son **d** y **e**. Por que habría de confiar en ellos?



Alice lo único que conoce es que Derek y Eric “verificaron” a Bob pero no tiene ni idea de cuanto confiar en ellos.

Alice si conoce a Carol.

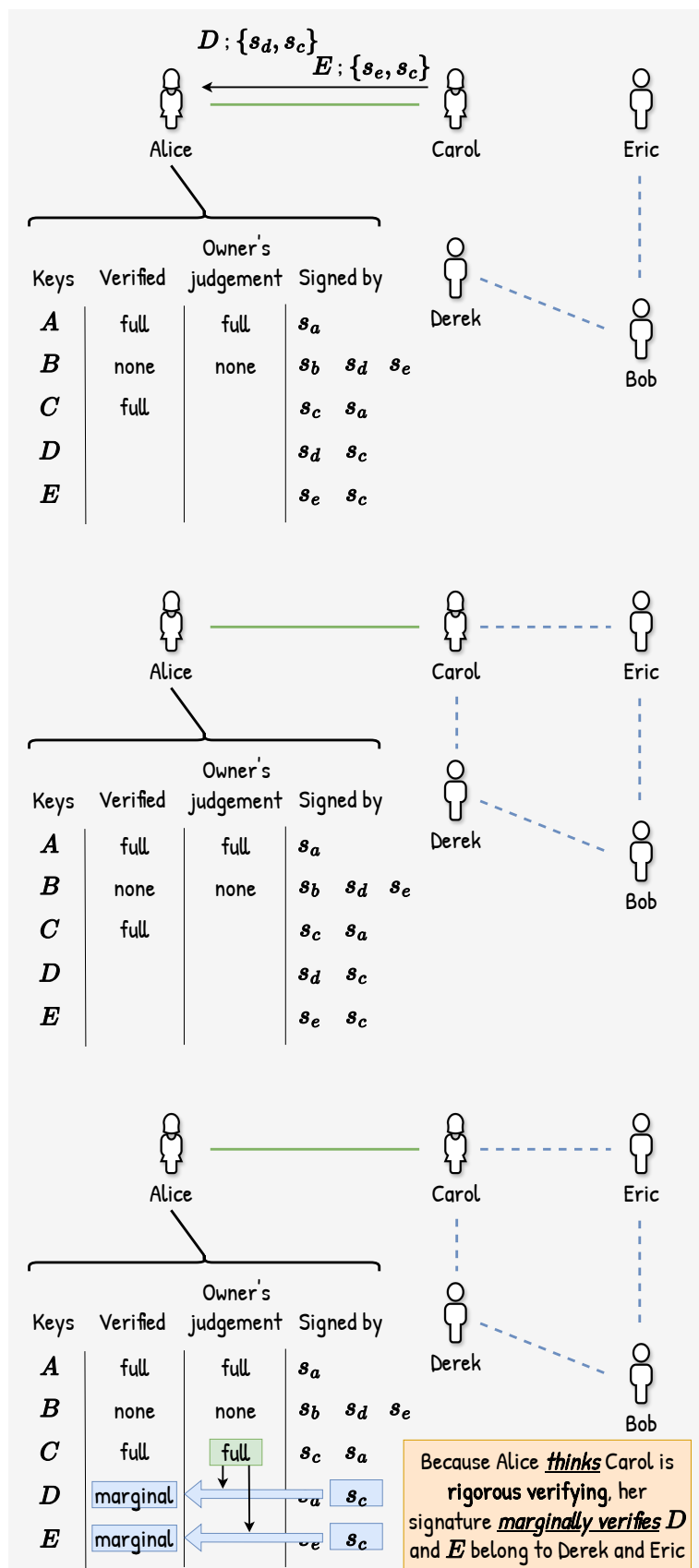
Intercambian sus públicas y quienes firmaron sus públicas. (Alice tiene **A** firmada solo por ella y Carol tiene **C** firmada solo por ella)



Alice verificó que **C** vino de Carol y verificó que Carol es Carol (la conoce personalmente de toda la vida).

Como Alice esta 100% segura de q Carol es Carol, se atreve a firmar con su privada la pública **C** como señal de “Yo Alice conozco a Carol y verifique q C es de ella”.

Ambas intercambian nuevamente sus públicas, ahora, con la firma de la otra.



Resulta ser que Carol tiene no solo las públicas de Derek y Eric sino tambien que ella se las firmo y se las pasa (junto con las firmas) a Alice.

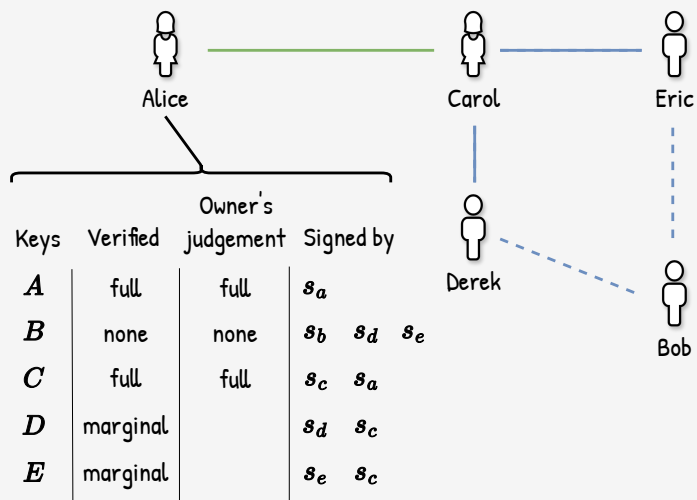
Si Carol les firmo las públicas D y E, es por que ella debe, de alguna manera conocer a Derek y Eric y habra, de alguna manera, verificado sus identidades.

No?

Alice debe hacerse esta pregunta: que tan confiable es el juicio de Carol? Es ella una persona meticulosa y firma solo lo que esta 100% segura? O es una persona que firma cuanto pública se le cruza sin verificar absolutamente nada?

Alice conoce a Carol y sabe q tiene un buen juicio y es responsable. Si Carol firmo las publicas de Derek y Eric, Alice confia completamente en Carol y asume que D y E estan verificadas.

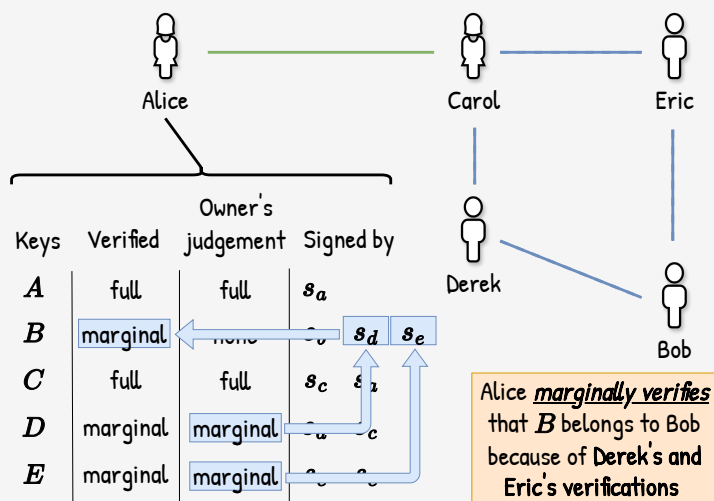
No completamente sino marginalmente (Alice deja un poquito de duda).



Alice ahora puede verificar la clave de Bob por que puede verificar las firmas de Derek y Eric.

Pero puede confiar en sus juicios? Carol le cuenta q Derek y Eric son responsables asi q Alice confia en sus juicios marginalmente.

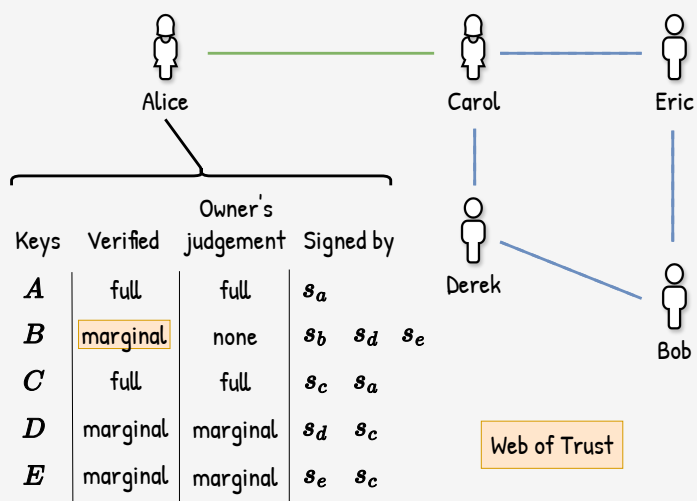
Si la clave de Bob estuviese firmada solo por uno de ellos, Alice no consideraría que es suficiente para verificar B. Pero como son 2 personas que hicieron la verificacion, Alice confía un poco más (marginalmente).



Alice, a pesar de no conocer a Bob, pudo obtener su pública y confiar en ella (y estar medianamente segura q B no es de Eve)

No hubo nadie de confianza. No hubo ninguna autoridad central, solo personas confiando en ellas formando no una cadena sino una red de confianza.

Este es el modelo que usa **gpg**. Lamentablemente es un esquema q escala poco y depende mucho del juicio de cada involucrado.



Takeaways

Symmetric
Cryptography

Fast

Easy

But relies on a
secret shared k so

Asymmetric
Cryptography

Slow

Not trivial!

It enables more
primitives

Signatures

Encryption

And *much* more

Certificates

Web of Trust

OK but with
weak points

But still, id verification
and pub key **trust** is
unresolved